

Problem 1.

Imagine a case when the signal handler `signal_handler()` is invoked while `some_function()` is in the middle of manipulating data structure (after acquiring `spin_lock(&ms->lock)`). Suddenly, `some_function()` gets interrupted. The signal handler attempts to acquire the same lock via `spin_lock(&ms->lock)`. Then, a deadlock happens as the signal handler spins forever waiting for a lock that can only be released by `some_function()`, but `some_function()` cannot continue because the signal handler has interrupted it and won't return.

Updated Version:

// We should block SIGINT inside *some_function* so that it doesn't occur while the function is running.

// <https://stackoverflow.com/questions/54352563/block-sigint-from-terminating-program>

// <https://www.ibm.com/docs/en/zos/2.5.0?topic=functions-sigprocmask-examine-change-thread>

```
struct shared_data_struct {
    int lock;
    /* Other data structures stuff */
} *ms;

/* signal_handler is the disposition of SIGINT */
void signal_handler(int sig)
{
    spin_lock(&ms->lock);
    /* Manipulate data structure */
    ms->lock=0;
}

int some_function(int a)
{
    sigset_t new_mask, old_mask;
    sigemptyset(&new_mask); // make an empty container.
    sigaddset(&new_mask, SIGINT); // add SIGINT to the list of signals to block.
    sigprocmask(SIG_BLOCK, &new_mask, &old_mask); // block SIGINT and save the previous blocked signal in old_mask.
    spin_lock(&ms->lock);
    /* Manipulate data structure */
    ms->lock=0;

    sigprocmask(SIG_SETMASK, &old_mask, NULL); // restore original signal mask.

    return something;
}
```